2009 IEEE International Symposium on Parallel and Distributed Processing with Applications

A Hardware-Software Cooperative Approach for the Exhaustive Verification of the Collatz Conjecture

Yasuaki Ito and Koji Nakano
Department of Information Engineering, Hiroshima University
lKagamiyama 1-4-1, Higashi-Hiroshima, 739-8527 JAPAN
yasuaki@cs.hiroshima-u.ac.jp, nakano@cs.hiroshima-u.ac.jp

***Abstract*—Consider the following operation on an arbitrary positive number: if the number is even, divide it by two, and if the number is odd, triple it and add one. The Collatz conjecture assert that, starting from any positive number n, repeated iteration of the operations eventually produces the value 1. The main contribution of this paper is to present hardware-software cooperative approach to verify the Collatz conjecture. The key idea of our approach is to sieve numbers n that produces 1 using a circuit implemented on an FPGA. The numbers that fail to be verified by overflow are reported to the host PC. The host PC verifies those numbers using unlimited bits operations by software. We have implemented 24 coprocessors on the Vertex II family FPGA XC2V3000-4. The experimental results show that our hardware-software cooperative approach can verify 2.89×10^9 64-bit numbers per second.**

***Keywords*-Hardware Algorithm; FPGA Implementation; block RAMs;**

I. Introduction

An FPGA (Field Programmable Gate Array) is a programmable VLSI in which a hardware designed by users can be embedded instantly. Typical FPGAs consist of an array of programmable logic blocks (or slices), memory blocks, and programmable interconnect between them. The logic block contains four-input logic functions implemented by a look up table and/or several registers. Using four-input logic functions, registers, and their interconnects, any combinational circuits and sequential logic can be implemented. Using design tools provided by FPGA vendors or third party companies, a hardware logic designed by users using hardware description languages can be embedded in FPGAs. It has been shown that a lot of computation can be accelerated using a circuit implemented in FPGAs [2], [3], [8], [9], [10].

The Collatz conjecture is a well-known unsolved conjecture in mathematics [5], [7], [13]. Consider the following operation on an arbitrary positive number:

even operation if the number is even, divide it by two, and

odd operation if the number is odd, triple it and add one.

The Collatz conjecture assert that, starting from any positive number, repeated iteration of the operations eventually produces the value 1. For example, starting from 3, we have the following sequence to produce 1.

$$3 \to 10 \to 5 \to 16 \to 8 \to 4 \to 2 \to 1$$

The main contribution of this paper is to present a hardware-software cooperative approach to verify the Collatz conjecture. More specifically, we have developed a hardware algorithm for exhaustive verification of the Collatz conjecture for a limited number of bits. We use an FPGA to implement this hardware algorithm as coprocessors for this approach. The key idea of our approach is to sieve numbers n verified that it produces 1 using coprocessors on an FPGA connected through PCI-bus. Each coprocessor has a 78-bit architecture and works as follows: First, it receives 32-bit number M from the host PC through the PCI-bus. For each 64-bit number m in $[M \cdot 2^{32}, (M+1) \cdot 2^{32} - 1]$, it verifies if iteration of the operations produce 1 starting from m by repeating the even and odd operations. The coprocessor reports number m to the host PC if the interim value has more than 78 bits and it fails to verify the conjecture. The host PC receives such number m, and perform the iteration of the operations with unlimited number of bits implemented by software. Let us show an example for the reader's benefit. Suppose that, from host PC, a 32-bit number M =0xF1234567 is given to the coprocessor on the FPGA. For each number m from $M \cdot 2^{32}$ (=0xF1234567000000) to $(M+1) \cdot 2^{32} - 1$ (=0xF1234567FFFFFFFF), the coprocessor repeats the even and the odd operations. Starting from m =0xF123456705A1CF67, the iteration of the operations produces an interim number 0xB59C3655454882EE20450CB8 which has more than 78 bits. Thus, the coprocessor detects the overflow of 78-bit register and reports the 32-bit LSB of m, 0x05A1CF67 to the host PC. The host PC verifies the Collatz conjecture for such m =0xF123456705A1CF67 by software.

This coprocessor approach makes sense because

- the hardware implementation on the FPGA is fast and low power consumption, but the number of bits for the operation is fixed,
- the software implementation on the PC is relatively slow and high power consumption, but the number of bits for the operation is unlimited.

978-0-7695-3747-4/09 $25.00 © 2009 IEEE
DOI 10.1109/ISPA.2009.35

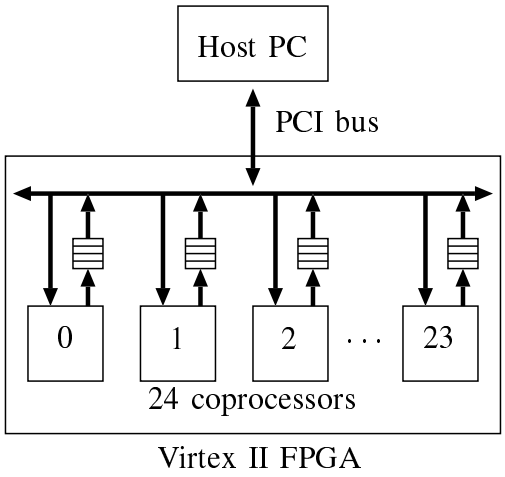

Figure 1. Our hardware-software cooperative architecture for verifying the Collatz conjecture

So, our approach finds counter example candidates m of the Collatz conjecture using the hardware implementation, and confirm it for such numbers m using the software implementation. Also, since the circuit for the coprocessor is small enough that we can implement 24 coprocessors on a Vertex II family FPGA XC2V3000-4 [6]. In this way, we can further accelerate the verification by hardware-software cooperative approach.

Figure 1 illustrates our hardware-software cooperative architecture for verifying the Collatz conjecture. We have used a Nallatech Xtreme DSP kit [11], which is a PCI board with Xilinx VirtexII family FPGA XC2V3000-4, We have implemented 24 coprocessor working independently in the FPGA. The FPGA and the host PC are connected though the PCI-bus. The host PC works as a server, which requests to verify the Collatz conjecture for 2^{32} numbers from $M \cdot 2^{32}$ to $(M+1) \cdot 2^{32} - 1$ for some 32-bit integer M to each coprocessor. The experimental results show that, for 64-bit numbers, each coprocessor can verify 1.20×10^8 numbers per second. Also, we can verify 2.89×10^9 numbers per second using our hardware-software cooperative architecture using 24 coprocessors.

In our previous paper [1], we have shown an architecture for the verification of the Collatz conjecture. This paper showed an implementation on a Xilinx Spartan-3AN family FPGA XC3S700AN-5, which uses 831 slices, seven 18-bit embedded multipliers, two 18Kbit block RAMs. The simulation and the timing analysis results for XC3S700AN-5 show that this architecture can verify 2^{16} numbers in 421μs. Thus, it can verify 1.55×10^8 numbers per second. However, it uses 128-bit registers, and does not check the overflow. Our new idea is to improve this architecture to work as a coprocessor, and to implement 24 coprocessors in a Virtex II family FPGA XC2V3000-4.

This paper is organized as follows. Section II shows basic ideas to accelerate the verification of the Collatz conjecture. In Section III, we show a basic hardware algorithm for the verification on the FPGA. Section IV shows how we have implemented our hardware algorithm as a coprocessor including the interface between the host PC and the FPGA, and how the software in the host PC works. Sections V and VI show the performance analysis and and the experimental results. Finally, Section VII offers the conclusions.

II. Accelerating the verification of the Collatz conjecture

The main purpose of this section is to show a hardware algorithm for accelerating the verification of the Collatz conjecture. The basic ideas of acceleration are shown in [7], [13].

The first idea is to terminate the operations before the iteration produces 1. Suppose that we have already verified that the Collatz conjecture is true for numbers less than n, and we are now in position to verify it for number n. Clearly, if we repeatedly execute the operations for n until the value is 1, then we can confirm that the conjecture is true for n. Instead, if the value becomes n' for some n' less than n, then we can guarantee that the conjecture is true for n because it has been proved to be true for n'. Thus, it is not necessary to repeat this operation until the value is 1, and we can terminate the iteration when, for the first time, the value is less than n.

The second idea is to perform several operations in one step. Consider that we want to perform the operations for n and let n_L and n_H be the least significant two bits and the remaining bits of n. In other words, $n = 4n_H + n_L$ holds. Clearly, the value of n_L is either 00, 01, 10, or 11. We can perform the several operations for n based on n_L as follows:

- $n_L = 00$: Since two even operations are applied, the resulting number is n_H.
- $n_L = 01$: First, odd operation is applied and the resulting number is $(4n_H + 1) \cdot 3 + 1 = 12n_H + 4$. After that, two even operations are applied, and we have $3n_H + 1$.
- $n_L = 10$: First, even operation is performed and we have $2n_H + 1$. Second, odd operation is applied and we have $(2n_H + 1) \cdot 3 + 1 = 6n_H + 4$. Finally, by even operation, the value is $3n_H + 2$.
- $n_L = 11$: First, odd operation is applied and we have $(4n_H + 3) \cdot 3 + 1 = 12n_H + 10$. Second, by even operation, the value is $6n_H + 5$. Again, odd operation is performed and we have $(6n_H + 5) \cdot 3 + 1 = 18n_H + 16$. Finally, by even operation, we have $9n_H + 8$.

For example, if $n_L = 11$ then we can obtain $9n_H + 8$ by applying 4 operations, odd, even, odd, and even operations in turn. Let A and B be tables as follows:

	A	B
00	1	0
01	3	1
10	3	2
11	9	8

Using these tables, we can perform the following table operation, which emulates several odd and even operations:

table operation For least significant two bits n_L and the remaining most significant bits n_H of the value, the new value is $A[n_L] \cdot n_H + B[n_L]$.

Let us extend the table operation for least significant two bits to more bits. For a number n and d (≥ 2), let n_L and n_H be the least significant d bits, that is, $n = 2^d n_H + n_L$. We call d is *the base bits*. Let the current value of n is $an_H + b$. Initially, $a = 2^d$ and $b = n_L$. We repeatedly perform the following rules for a and b.

even rule If both a and b are even, then divide them by two.

odd rule If b is odd, then triple a, and triple b and add one.

These two rules are applied until no more rules can be applied, that is, until a is odd and b is even. It should be clear that, even and odd rules correspond to even and odd operations of the Collatz conjecture. If i even rules and j odd rules applied, then the value of a is $2^{d-i}3^j$. Thus, exactly d even rules are applied until the termination. After the termination, we can determine the value of elements in tables A and B such that $A[n_L] = a$ and $B[n_L] = b$. Using tables A and B, we can perform the table operation for d bits n_L, which involves d even operations and zero or more odd operations. In this way, we can accelerate the operation of the Collatz conjecture. In our previous paper [1], we have implemented for various numbers of bits of n_L. Our implementation results show that the performance is well balanced when the number of bits of n_L is 10.

The third idea to accelerate the verification of the Collatz conjecture is to skip numbers n such that we can guarantee that the resulting number is less than n after the table operation. For example, suppose we are using two bit table and $n_H > 0$. If $n_L = 00$ then the resulting value is n_H, which is less than n. Thus, we can skip the table operation for n if $n_L = 00$. If $n_L = 01$ then the resulting value is $3n_H + 1$, which is always less than $n = 4n_H + 1$, and we can skip the table operation. Similarly, if $n_L = 10$ then we can skip the table operation. On the other hand $n_L = 11$ then the resulting value is $9n_H + 8$, which is always larger than n. Therefore, the Collatz conjecture is guaranteed to be true whenever $n_L \neq 11$, because it has been verified true for numbers less than n. Consequently, we need to execute the table operation for number n such that $n_L = 11$.

We can extend this idea for general case. For least significant d bits n_L, we say that n_L is not *mandatory* if the value of a is less than 2^d at some moment while even and odd rules are repeatedly applied. We can skip the verification for non mandatory n_L. The reason is as follows: Consider that for number n, we are applying even and odd rules. Initially, $a = 2^d$ and $b \leq 2^d - 1$ hold. Thus, while even and odd rules are applied, $a > b$ always hold. Suppose that $a \leq 2^d - 1$ holds at some moment while the rules are applied. Then, the current value of n is

$$an_H + b < an_H + a \leq (2^d - 1)n_H + a = 2^d n_H < n.$$

It follows that, the value is less than n when the corresponding even and odd operations are applied. Therefore, we can omit the verification for numbers that have no mandatory least significant bits.

For least significant d bit number, we use table S to store the mandatory least significant bits. Let s_d be the number of such mandatory least significant bits. Using these table, we can write a verification algorithm as follows:

for $m_H = 1$ **to** $+\infty$ **do**
  for $i = 0$ **to** $s_d - 1$ **do**
    begin
      $m_L = S[i]$;
      $n = m = 2^d m_H + m_L$;
      while($n \geq m$) **do**
        begin
          Let n_L be the least significant d bits and n_H be the remaining bits.
          $n = A[n_L] \cdot n_H + B[n_L]$;
        end
    end

For the benefit of readers, we show A, B, and S for 4 base bits in Table I. From $s_4 = 3$, we have 3 mandatory least significant bits out of 16.

Table I
TABLES A, B, AND S FOR LEAST SIGNIFICANT 4 BITS.

	A	B	S
0000	1	0	0111
0001	9	1	1011
0010	9	2	1111
0011	9	2	-
0100	3	1	-
0101	3	1	-
0110	9	4	-
0111	27	13	-
1000	3	2	-
1001	27	17	-
1010	3	2	-
1011	27	20	-
1100	9	8	-
1101	9	8	-
1110	27	26	-
1111	81	80	-

III. An Architecture for the verification on an FPGA

The main purpose of this section is to show an architecture for verifying the Collatz conjecture using tables A, B, and S. The preliminary results of FPGA implementation are shown in our previous paper [1].

Let d be the base bits. Each of the tables A and B can be stored in a ROM of size $2^d \times a_d$, where a_d is the number of bits necessary to store the values of A. From Table I, $a_4 = 7$ because the maximum value is $81 < 2^7$. To store table S, a ROM of size $s_d \times d$ is used. Let w be the maximum number of bits of n. Clearly, n_H has $w - d$ bits and n_L has d bits. Thus, to compute the product $A[n_L] \cdot n_H$, we use an $a_d \times (w - d)$-bit multiplier. After that, to compute the sum $A[n_L] \cdot n_H + B[n_L]$, a w-bit a_d-bit adder is used.

The ROM can be implemented by block RAMs in the FPGA. Block RAMs in most FPGAs including Virtex II Family FPGA used in our implementation support synchronous read and synchronous write [6]. In the synchronous operation, the address bits given to an address port is written in an internal address register at the rising edge of the clock. After that, the data specified by the address register can be read from the data output port. Thus, one clock cycle necessary to read the data in a synchronous read block RAM. A naive implementation of the ROM using the block RAM needs two clock cycles to update the value of n_H and n_L by next value $A[n_L] \cdot n_H + B[n_L]$. One clock cycle is necessary to read the value of $A[n_L]$ and $B[n_L]$ stored in the block RAMs. After that, additional one clock cycle is necessary to store that value of $A[n_L] \cdot n_H + B[n_L]$ to registers for n_H and n_L. As we are going to show next, the update of n_H and n_L can be done in one clock cycle by careful implementation.

Figure 2 shows an illustration of our architecture for verifying the Collatz conjecture, which performs the update of n_H and n_L in every clock cycle. The address registers for tables A and B are used to store n_L as in Figure 2, while the value of n_H is stored in a register. Two counters are used to store the current values of m_H and i. The value of $S[i]$ is computed from i and stored in m_L. In the same time, $S[i]$ is also stored in the address register for tables A and B. From the value of the address register, $A[n_L]$ and $B[n_L]$ are obtained. After that, $A[n_L] \cdot n_H + B[n_L]$ is computed using a multiplier and an adder. This value is compared with the value of m stored in the register. If $n \geq m$ then the resulting value is stored in the register for n_H and the address register for n_L. Otherwise these registers are updated by next values of m_H and m_L. In this way, the values of n_H and n_L can be updated in one clock cycle.

IV. Our hardware-software cooperative approach

The main purpose of this section is to present the details of implementation of our hardware-software operative approach.

We have used 24 coprocessors as illustrated in Figure 1. We modify the architecture in Figure 2 to use as a coprocessor in Figure 1. The readers should refer Figure 3 for illustrating the interface between the host PC and our coprocessor architecture. Our coprocessor architecture takes 10 base bits, and thus, tables A and B has 1024 16-bit words each. It has 64 mandatory least significant bits out of 1024 least significant bits. Thus, the counter for table S needs 6 bits to indicate one of the 64 mandatory least significant bits. Each of the coprocessors receives 32-bit integer M from the host PC, and performs the exhaustive verification for the Collatz conjecture for numbers from $M \cdot 2^{32}$ to $(M+1) \cdot 2^{32} - 1$. In other words, the verification is performed for 64-bit numbers. This is reasonable because working projects for the Collatz conjecture are currently checking 60-bit numbers [12] and 63-bit numbers [4]. Suppose that the coprocessor find that the interim value is overflow for the initial value m. The coprocessor reports the least significant 32-bit of m through the buffer with four 32-bit registers if the overflow is detected. After the host PC receives this 32-bit value, it verifies the Collatz conjecture for m.

Figure 4 illustrates a part of the circuit to compute $A[n_L] \cdot N_H + B[n_L]$ and to detect the overflow. The value of n_H is stored in 68-bit register, and that of n_L is stored in the block RAMs for tables A and B. Note that, since our architecture has 10 base bits, n_L has 10 bits and the output values $A[n_L]$ and $B[n_L]$ of tables A and B have 16 bits. The product $A[n_L] \cdot n_H$ is computed by a 68×16-bit multiplier. The sum $A[n_L] \cdot n_H + B[n_L]$ is computed by an $84 + 16$-bit adder. If the resulting value of the sum is larger than 2^{78}, the overflow is detected. If this is the case, the least significant 32 bits of the current value of m is transferred to the buffer in Figure 3.

The host PC works as a server of the coprocessors as follows: Suppose that we need to verify the Collatz conjecture for all 64-bit numbers. The host PC sends $0, 1, 2, \ldots, 23$, to the 24 coprocessors respectively. The host PC always checks the buffers of the coprocessors. Consider that the host PC finds a 32-bit number m' stored in one of the buffers of the coprocessor and the host PC has sent request of 32-bit number M to the coprocessor. It follows that the coprocessor detect the overflow for the initial value $m = M \cdot 2^{32} + m'$. Thus, the host start start the verification for m using the unlimited bits operations. If the host PC finds that one of the coprocessors finishes the verification for 2^{32} numbers, it sends next number 24 to the processor. Similarly, if the host PC finds a processor that finishes the verification, it sends next numbers 25, 26, ..., $2^{32} - 1$.

The unlimited bits operations by software are implemented by a linked list with each element taking a 32-bit unsigned integer. Hence, a linked list with n elements can represent $32n$-bit integer. Since n is unlimited, the linked list can represent any large number. Further, it is not difficult to implement the even and the odd operations for a linked list

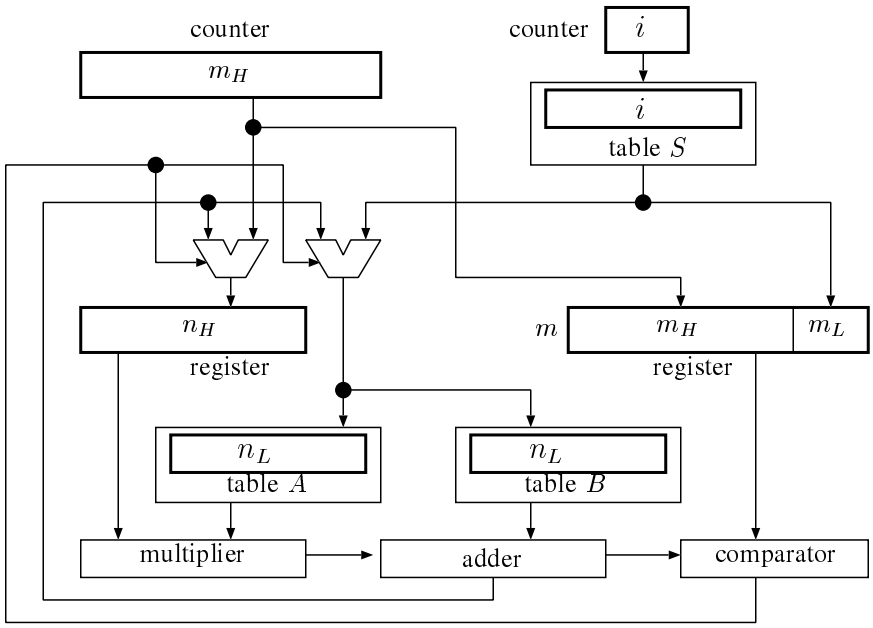

Figure 2. Our architecture for verifying the Collatz conjecture

in an obvious way. In particular, by the odd operation, we may need more number of bits. If this is the case, an element with 32-bit number is added to the linked list to increase the number of bits.

V. The performance analysis

Let us evaluate the hardware resources in the FPGA to implement coprocessors. A coprocessor uses a 68×16-bit multiplier. The Virtex II FPGA has 18×18-bit multipliers, which support the multiplication of two 17-bit unsigned integers. Thus, the product of 68-bit and 16-bit unsigned integers can be implemented using four 18×18-bit multipliers. Tables A and B have 1024 16-bit words each. Hence, each of them can be implemented in one 18k-bit block RAM in the Virtex II FPGA. Table S has 64 10-bit words, which can also be implemented in one 18k-bit block RAM. Thus, each coprocessor uses three 18k-bit block RAMs for tables A, B, and S. Further, since the block RAM in the Virtex II FPGA is dual port, that is, it has two address ports and can be read the data of two addresses, which can be distinct, in the same time. Hence, two coprocessors can share the three 18k-bit block RAMs for three tables. Consequently, 24 coprocessors uses 96 18×18-bit multipliers and 36 18k-bit block RAMs. Since our target is XC2V3000 has 96 18×18-bit multipliers and 96 18k-bit block RAMs, our implementation is feasible.

Let us evaluate how often we have the overflow. Suppose that the even and the odd operations are performed for a p-bit number n. Note that $2^{p-1} \leq n < 2^p$ holds. Let $M(n)$ denote the maximum number until n produces, for the first time, a number less than n during the operations. For example, $M(3) = 16$ holds. Suppose that we use b-bit architecture for the even and the odd operations. The verification of n is overflow if $M(n) \geq 2^b$. Let $V(p, b)$ be the set of such number n, that is $V(p, b) = \{n \mid 2^{p-1} \leq n < 2^p \text{ and } M(n) \geq 2^b\}$. Then, the ratio $R(p, b)$ of the overflow of p-bit numbers on b-bit architecture is

$$R(p, b) = \frac{|V(p, b)|}{2^{p-1}}.$$

Let us clarify the ratio $R(p, b)$ for $p = 58, 59, \ldots, 64$, and $b = 66, 67, \ldots, 90$. For this purpose, we generate a p-bit number 10^8 times at random, and see if $M(n) \geq 2^b$. Figure 5 shows the ratio $R(p, b)$ of overflow for a p-bit integer on the b-bit architecture. This figure shows that the ratio $R(p, b)$ rapidly decreases by increasing the value of b. Recall that the parameters $p = 64$ and $b = 78$ are used in our coprocessors. From the figure, we can see $R(64, 78) = 2.9 \times 10^{-5}$, which is small enough.

VI. Experimental results

We have implemented and evaluated the performance of our hardware-software cooperative approach on Xilinx Virtex II family FPGA (XC2V3000-4), which has 28672 slices, 96 18-bit multipliers, and 96 18k-bit block RAMs. For logic synthesis, we have used XST with ISE Foundation 10.1. The implemented circuit on the FPGA consists of 24 coprocessors and the interface between Host PC and FPGA. The implementation uses 8848 slices and 96 18-bit multipliers, and 36 18k-bit block RAMs. The timing analysis by ISE Foundation 10.1 reported that our implementation

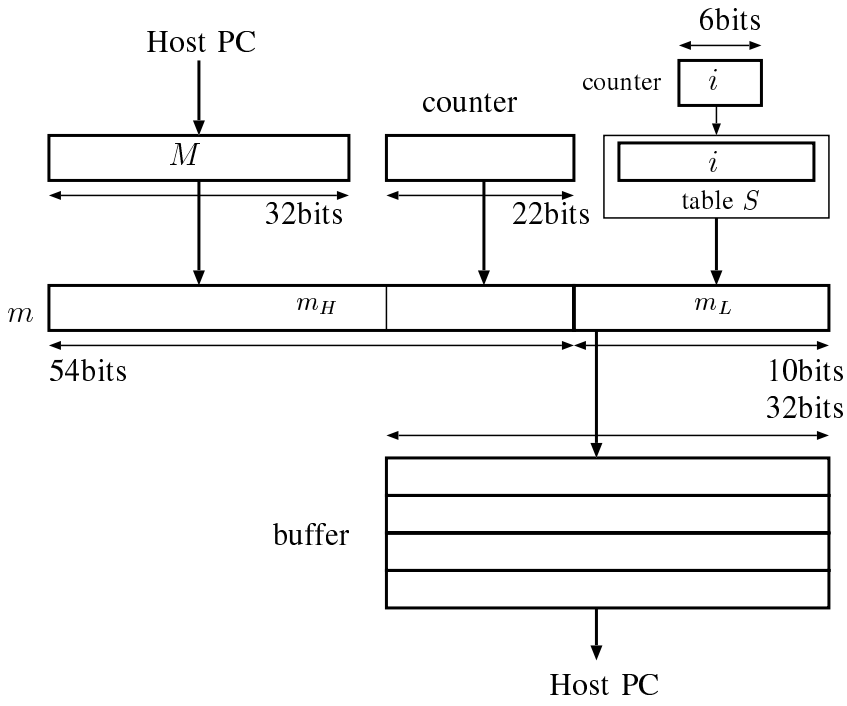

Figure 3. The interface between the host PC and the coprocessor

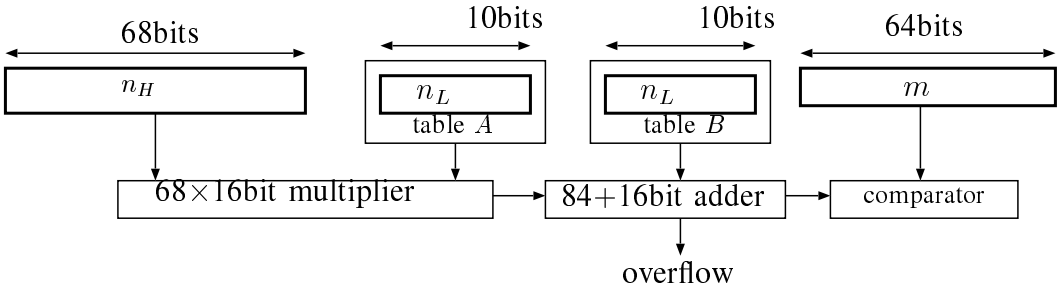

Figure 4. The circuit to compute $A[n_L] \cdot N_H + B[n_L]$ and to detect the overflow

runs in 40.12MHz. Thus, we set the clock frequency the programmable oscillator on the FPGA board to 40MHz.

The computing time is evaluated by verifying the Collatz conjecture for the 64-bit numbers in 240 sections. Each section has 2^{32} numbers in $[M \cdot 2^{32}, (M+1) \cdot 2^{32} - 1]$, where M is a 32-bit number that is generated at random. Therefore, we verified $240 \times 2^{32} \approx 1.0 \times 10^{12}$ 64-bit numbers. For the purpose of showing the goodness of our hardware-software cooperative approach, we have implemented a software approach. We measured the performance of the software approach on an IBM PC-compatible (Xeon X5355 2.66GHz processor with 10GB of memory) using Linux OS (64bit). Since enough size of memory is available in the software approach, we use larger tables A, B, and C. More specifically, the sizes of tables A and B are 65536×26 and the size of table S of sizes 2114×16. The computing time of the software approach is also evaluated by verifying the Collatz conjecture for the same 64-bit numbers in 240 sections.

Table II shows the computing time of our hardware-software cooperative approach and its corresponding software approach for verifying approximately 1.0×10^{12} 64-bit numbers. As shown in the table, a speed-up factor of more than 15 is experimented and our hardware-software cooperative approach verified 2.89×10^9 numbers per second. In the hardware-software cooperative approach, there are 5656255 overflow-numbers. The number of overflow is much smaller than the number of verified numbers, and those numbers are verified by software on the host PC. Therefore, almost numbers are sieved using the circuit on the FPGA.

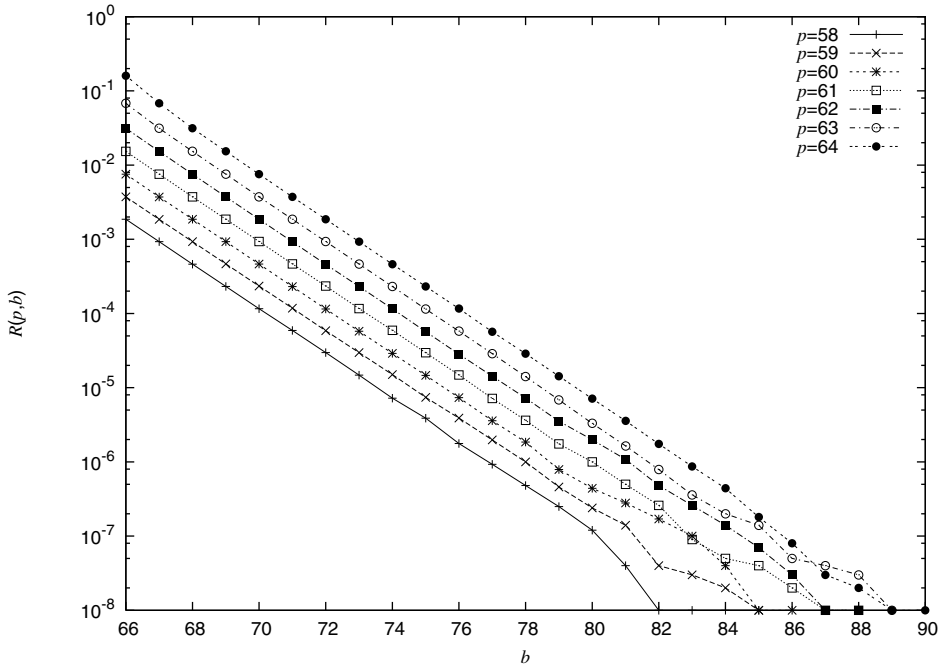

Figure 5. The ratio $R(p, b)$ of overflow for a p-bit integer on the b-bit architecture

Table II
THE COMPUTING TIME FOR VERIFYING COLLATZ CONJECTURE

	Computing Time	The number of verified numbers per second	Speed-up
Software approach	5517.67 sec	1.87×10^8	–
Hardware-software cooperative approach	357.17 sec	2.89×10^9	15.45

VII. CONCLUSIONS

We have presented a hardware-software cooperative approach that verifies the Collatz conjecture. The key idea of our approach is to sieve numbers n verified that it produces 1 using a circuit on an FPGA. The numbers that fail to be verified by overflow are reported to the host PC. The host PC verify those numbers using unlimited bits operations by software.

We have implemented 24 coprocessors on the Vertex II family FPGA XC2V3000-4. The experimental results show that our hardware-software cooperative approach can verify 2.89×10^9 numbers per second, and attains a speed-up factor of more than 15 over the software approach.

REFERENCES

[1] F. An and K. Nakano. An architecture for veryfing collatz conjecture using an fpga. In *Proc. of the International Conference on Applications and Principles of Informatin Science*, pages 375–378, 2009.

[2] J. L. Bordim, Y. Ito, and K. Nakano. Accelerating the CKY parsing using FPGAs. *IEICE Transactions on Information and Systems*, E86-D(5):803–810, May 2003.

[3] J. L. Bordim, Y. Ito, and K. Nakano. Instance-specific solutions to accelerate the CKY parsing for large context-free grammars. *International Journal on Foundations of Computer Science*, pages 403–416, 2004.

[4] T. O. e Silva. Computational verification of the 3x+1 conjecture. http://www.ieeta.pt/ tos/3x+1.html.

[5] T. O. e Silva. Maximum excursion and stopping time record-holders for the $3x + 1$ problem: Computational results. *Mathematics of Computation*, 68(225):371–384, Jan. 1999. Up to date computational results can be found at `http://www.ieeta.pt/~tos/3x+1.html`.

[6] X. Inc. *Virtex-II Platform FPGAs: Complete Data Sheet*, 2003.

[7] J. C. Lagarias. The 3x+1 problem and its generalizations. *The American Mathematical Monthly*, 92(1):3–23, 1985.

[8] K. Nakano and E. Takamichi. An image retrieval system using FPGAs. *IEICE Transactions on Information and Systems*, E86-D(5):811–818, May 2003.

[9] K. Nakano and K. Wada. Integer summing algorithms on reconfigurable meshes. *Theoretical Computer Science*, 197:57–77, 1998.

[10] K. Nakano and Y. Yamagishi. Hardware n choose k counters with applications to the partial exhaustive search. *IEICE Trans. on Information & Systems*, 2005.

[11] Nallatech. *Xtreme DSP Development Kit User Guide*, 2002.

[12] E. Roosendaal. On the 3x + 1 problem. http://www.ericr.nl/wondrous/index.html.

[13] E. W. Weisstein. Collatz problem. From MathWorld–A Wolfram Web Resource. http://mathworld.wolfram.com/CollatzProblem.html.